2026-03-31

Modernising Integer Partitions: Implementing Standard Iterators for the partitions Package

Sam Kamperis
Oxford Brookes University

Robin Hankin
University of Stirling

Executive Summary

Overview

The `partitions` package (Robin K. S. Hankin 2006) is a widely-used R utility for enumerating integer partitions and related combinatorial structures. With over 1.3 million downloads, 6000–8000 monthly users, and 36 dependencies, it is critical R infrastructure. However, the package suffers from two key limitations: (1) bulk-generation APIs cause catastrophic memory exhaustion for large problems, and (2) the existing iterator API is non-standard, preventing seamless integration with modern R ecosystems (`foreach`, `itertools` (Weston and Wickham 2014), `parallel`).

This proposal seeks to modernise the `partitions` package by implementing the standard S3 `iterators` protocol (Analytics and Weston 2022), enabling lazy evaluation, efficient memory usage, and interoperability with parallel computing frameworks. We will create a new S3 class `partitions_iter` with `nextElem` methods for the core enumeration functions, along with comprehensive documentation and vignettes demonstrating practical use cases.

Project Scope

Primary Deliverable: An S3 `partitions_iter` class providing scalable, lazy iteration that conforms to the `iterators` interface, enabling direct integration with `foreach()` and `itertools` (Weston and Wickham 2014).

Secondary Deliverables:

- Comprehensive Quarto vignette with 5+ worked examples: serial iteration, `foreach()` parallelisation, `itertools` chaining, memory-efficiency comparisons.
- Full test suite covering concurrent iterator behaviour, edge cases, and GitHub issue resolution.
- CRAN-ready code and final review with Robin Hankin.

Budget & Timeline

We request **$11,989.98** to fund **124 hours** of development. This is a joint costing from Oxford Brookes University and the University of Stirling.

Sam Kamperis (Oxford Brookes) would be allocated 80 hours for the development. Robin Hankin (Stirling) would be allocated 44 hours.

Item	Cost (USD)
Sam Kamperis 80 hours	**$5,206.27**
Robin Hankin 44 hours	**$6,783.71**
Total	**$11,989.98**

Team

The project is led by the grant applicant (9.7% FTE over 6 months) with **Robin Hankin**, the original `partitions` package author, as a collaborator providing technical guidance, code review, and CRAN liaison.

Expected Impact

This modernisation unlocks new use cases: practitioners can now leverage the package seamlessly within modern R functional pipelines and parallel frameworks, enabling applications such as high-throughput research tasks in operations research, cryptography, and scientific computing. The new iterator interface allows direct integration with tools like `foreach`, `future`, and `itertools`, supporting reproducible research, teaching, and complex computational workflows that were previously impractical. Previously, workarounds required verbose, error-prone custom code, discouraging adoption in teaching and research, and the lack of standard iterator support prevented the package's use in parallel and distributed computing, limiting its utility in modern data science workflows.

Signatories

Project team

Sam Kamperis Github
Robin Hankin Github

Contributors

Consulted

James Maunder
Fridolin Wild Github
Inna Skarga-Bandurova Github

The Problem

What Is the Problem?

The `partitions` package (Robin K. S. Hankin 2006) provides two distinct APIs for enumerating partitions:

1. **Bulk-generation functions** (`parts()`, `restrictedparts()`, `diffparts()`): Generate all partitions in memory as a matrix and return them at once.
2. **Stepwise iteration** (`firstpart()` + `nextpart()`, etc.): Users write manual `while` loops, maintaining state vectors and calling low-level stepping functions.

Both approaches have critical limitations.

Combinatorial Explosion & Memory Crashes

For even moderate values of n, the number of partitions grows explosively. The partition count function $p(n)$ grows as:

$$p(n) \sim \frac{1}{4n\sqrt{3}} \exp\left(\pi\sqrt{\frac{2n}{3}}\right)$$

Concretely: $p(50) \approx 204$ million partitions; $p(100) \approx 190$ quadrillion. Users attempting `parts(50)` or larger will exhaust RAM and crash their R session. This is not a minor inconvenience, it is a **hard blocker** for realistic combinatorial research.

Non-Standard Iterator Protocol

The `firstpart()`/`nextpart()` API does not conform to the R ecosystem's standard iterator interface defined by the `iterators` package (Analytics and Weston 2022). Users cannot:

- Pass partition iterators directly to `foreach()` without custom wrappers.
- Chain partition iterators with `itertools` utilities like `ichain()`, `ifilter()`, or `izip()` (Weston and Wickham 2014).
- Use partition enumeration in standard parallel frameworks designed for iterators.

Instead, users write verbose, error-prone `while` loops:

```
part <- firstpart(20)
results <- list()
while (!islastpart(part)) {
  results[[length(results) + 1]] <- process(part)
  part <- nextpart(part, check=FALSE)
}
```

This pattern is tedious, hard to maintain, and incompatible with lazy evaluation and parallelisation idioms.

Who Is Affected?

With **1.3 million total downloads** and **6,000–8,000 monthly users**, the `partitions` package serves:

- **Researchers** in combinatorics and number theory performing exhaustive enumeration.
- **Statisticians** using partitions for resampling and bootstrap methods.
- **Operations researchers** solving integer partition problems.
- **Parallel computing practitioners** seeking to distribute combinatorial tasks across clusters.

Why Is This a Problem?

1. **Ecosystem Misalignment:** Modern R functional programming (via `foreach`, `itertools`, `furrr` (Vaughan and Dancho 2022)) assumes standard iterator interfaces. `partitions` is an exception, creating friction and requiring custom adapters.
2. **Memory Degradation:** Researchers working with realistic problem sizes cannot use convenient bulk-generation APIs and must resort to manual state management, introducing bugs and reduced code clarity.
3. **Concurrency Barriers:** Parallel frameworks cannot transparently distribute work over partition enumerations because the non-standard API provides no standard way to yield values on demand.

What Will Solving This Enable?

Implementing standard iterators will:

- **Enable lazy evaluation:** Users iterate over arbitrarily large partition streams without materialising all values in memory.
- **Unlock parallel workflows:** Standard iterators integrate seamlessly with `foreach()`, `parallel::makeCluster()`, and `furrr`.
- **Improve interoperability:** Partition iterators can be composed with other `itertools` operations (filtering, zipping, chaining).
- **Reduce cognitive load:** Users write familiar, standard R code instead of custom loops.

Existing Work & Precedent

Standard iterator support is well-established throughout the R ecosystem:

- The `iterators` package (Analytics and Weston 2022) defines the S3 `iter` class and `nextElem` generic method.
- The `itertools` package (Weston and Wickham 2014) provides composable combinators (`chain`, `filter`, `zip`) that work with any standard iterator.
- Packages like `foreach` and `furrr` assume standard iterator interfaces for parallel iteration.

The `partitions` package simply needs to align with these established conventions. The existing `firstpart()`/`nextpart()` API will remain for backwards compatibility; we will **add** (not replace) the standard interface.

Competitive Landscape

Several other R packages provide combinatorial generation and partition enumeration. Understanding their scope clarifies the gap this proposal addresses.

`RcppAlgos` (Wood 2026b, 2026a) provides highly optimised C++ routines for combinatorics, integer partitions, and number theory. Its `partitionsGeneral()` and `partitionsIter()` functions are among the fastest available in R, as documented in published benchmarks (Wood 2026a). However, `RcppAlgos` exposes partitions through a custom S4 class (`Partitions-class`) with proprietary iteration methods (`@nextIter`, `@prevIter`, `@randomAccess`) that do not implement the standard `iterators` S3 protocol. Passing a `RcppAlgos` iterator to `foreach()` or composing it with `itertools` combinators requires bespoke wrapper code — the same friction this proposal resolves for `partitions`. Additionally, `RcppAlgos` focuses on numeric constrained-sum partitions and does not cover the set-partition (`setparts()`), block-partition (`blockparts()`), or multiset-partition semantics unique to `partitions`.

`arrangements` (Lai 2020) provides memory-efficient generators and iterators for permutations, combinations, integer partitions, and compositions, with its own `ipartitions()` function. Like `RcppAlgos`, `arrangements` exposes a package-specific API (`getnext()`, `collect()`) rather than the standard `nextElem` interface, so partition iterators cannot be passed directly to `foreach` or `itertools` without custom adaptation.

Our approach is complementary, not competitive. This proposal does not aim to replicate the compiled C++ performance of `RcppAlgos` or `arrangements`. Instead, it modernises the *interface layer* of the `partitions` package — bringing its unique partition types in line with the standard `iterators` protocol so they integrate natively with `foreach`, `furrr`, and `itertools`, an interoperability benefit that neither alternative currently provides.

The Proposal

Overview

We will modernise the `partitions` package (Robin K. S. Hankin 2006) by implementing the standard S3 `iterators` protocol (Analytics and Weston 2022), enabling lazy, memory-efficient enumeration of partitions with seamless integration into the modern R ecosystem. The core deliverable is a new `partitions_iter` S3 class providing the `nextElem` method, allowing users to pass partition enumerations directly into `foreach()` loops, `itertools` (Weston and Wickham 2014) combinators, and parallel frameworks.

The solution preserves all existing APIs for backwards compatibility while adding a thin, well-tested layer that conforms to established iterator conventions. This approach minimises risk: we leverage the battle-tested `firstpart()`/`nextpart()` enumeration logic rather than replacing it, and we add comprehensive documentation and examples to guide users.

Detail

Minimum Viable Product

A working MVP consists of:

1. **S3 class `partitions_iter`** with fields `state` (current partition vector), `n` (the target integer), and `type` (enumeration type: "unrestricted", "restricted", "distinct", etc.).

2. **Method `nextElem.partitions_iter()`** that calls the appropriate stepping function (`nextpart()`, `nextdiffpart()`, etc.), returns the current state, advances state via the `<<-` operator, and throws `'StopIteration'` when exhausted.
3. **Factory functions** `iparts()`, `irestrictedparts()`, `idiffparts()` that return an iterator (S3 object) for lazy enumeration.
4. **Basic `testthat` (Wickham 2011) tests** verifying that iterators produce the same results as bulk-generation functions, and that multiple iterators coexist without state collision.

Example usage:

```
library(partitions)
library(foreach)

# Serial enumeration
it <- iparts(15)
while (!islastpart(it$state)) {
  print(nextElem(it))
}

# Parallel iteration with foreach
results <- foreach(part = iparts(20), .combine = c) %dopar% {
  sum(part)  # Example: sum each partition
}
```

Architecture

R-level: - `iparts(n)` → creates an S3 object of class `c("partitions_iter", "abstractiter", "iter")` - Object stores `state` (current partition vector), `n`, and `type` - `nextElem.partitions_iter()` closure uses `<<-` to mutate state in the enclosing environment

C-level: - No changes required; existing `c_nextpart()`, `c_nextrestrictedpart()`, etc. are reused as-is - These functions already provide re-entrant state mutation (each partition vector is independent)

Integration: - The `iter()` generic method (inherited from `iterators` package) returns the iterator itself - `foreach()` and `itertools` function detect the `iter` class and call `nextElem()` in a loop - State isolation is automatic: each call to `iparts()` creates a new closure with its own `state` variable

Benchmarking & Efficiency Methodology

To substantiate the memory-efficiency claims in this proposal and to validate iterator call overhead, we will produce a reproducible benchmark suite included in the package vignette and deposited in `inst/benchmarks/`. Benchmarks focus on two dimensions most relevant to the proposal goals. We do not benchmark against `RcppAlgos` (Wood 2026b) or `arrangements` (Lai 2020) for raw generation throughput: those packages use compiled C++ routines operating in a different performance tier, and raw speed is not the objective here.

Memory efficiency — bulk vs. lazy generation:

Target n	Partitions $p(n)$	Bulk `parts(n)`	Lazy `iparts(n)`
20	627	Full matrix in memory	O(n) peak, constant
30	5,604	Full matrix in memory	O(n) peak, constant
50	204,226	Typically exceeds available RAM	Viable
100	$\approx 1.9 \times 10^{14}$	Infeasible	Viable

Memory profiling will use `bench::mark(memory = TRUE)` (Hester and Vaughan 2023) together with RSS measurements via `profmem`. For $n \geq 50$, bulk generation is expected to crash or exhaust available memory, providing direct empirical evidence for the memory-safety motivation of this work.

Iterator call overhead — new interface vs. existing stepping API:

We will compare the per-step time of `nextElem(iparts(n))` against the equivalent `nextpart(part, check = FALSE)` manual loop. Both paths invoke the same underlying C-level enumeration logic, so overhead from the S3 wrapper layer is expected to be negligible. Results will be reported using `bench::mark()` over at least 30 repetitions per condition.

Reproducibility: Benchmark scripts and raw results will be committed to `inst/benchmarks/` and rendered in a vignette (memory-efficiency comparison).

Assumptions

1. **The C-level enumeration logic is correct and mature.** We are not reimplementing it; only wrapping it.
2. **The `iterators` package API is stable.** (It is; the package is mature and widely used.)
3. **Users have installed `iterators` as a dependency.** (We will add it to `DESCRIPTION` with `Imports: iterators`.)
4. **Multiple iterators can coexist without state collision.** (They can, because each iterator closure has its own environment.)

Failure Modes & Mitigation

Failure Mode	Likelihood	Mitigation
Iterator state mutations are slow due to copy-on-modify semantics	Low	Partition vectors are relatively small ($<$ 1000 elements typically); overhead is negligible.
Users expect `stop('StopIteration')` instead of `error()`	Low	`nextElem()` from `iterators` package already standardises this; we follow the convention.
Parallel workers cannot access iterator closures	Low	Each worker receives a serialised copy of the iterator object (state vector, n, type); no shared mutable state.

Failure Mode	Likelihood	Mitigation
Restrictedparts edge cases fail (e.g., `m > n`)	Medium	Comprehensive unit tests; review existing test coverage; coordinate with Robin for edge-case validation.

External Dependencies

- **`iterators`** (CRAN): S3 class definitions and `nextElem` generic. Must be added to `Imports:` in `DESCRIPTION`.
- **`foreach`** (CRAN, optional): User convenience for parallel enumeration. Not a hard dependency.
- **`itertools`** (CRAN, optional): User convenience for composing iterators. Not a hard dependency.
- **`testthat`** (CRAN, existing dev dependency): Test infrastructure.

All dependencies are mature, actively maintained, and widely adopted. No risk of ecosystem instability.

Evidence of Impact

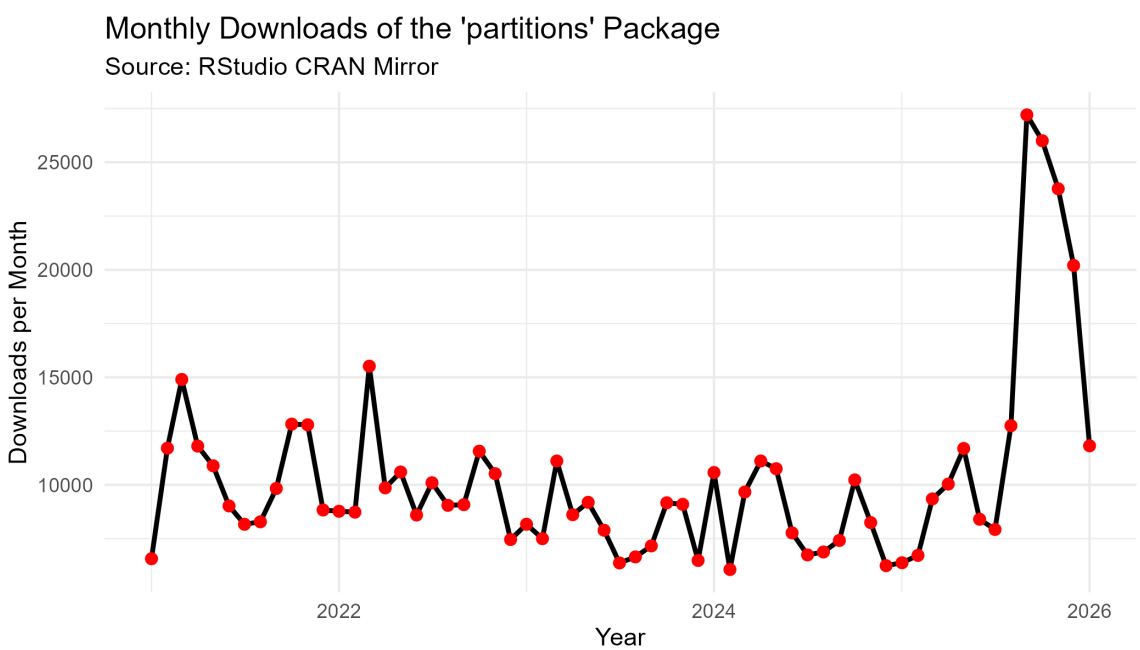

Figure 1: Monthly Downloads of the partitions package

The plot above demonstrates the `partitions` package's significant and sustained user adoption. With ~**1.3 million cumulative downloads** and **6,000–8,000 monthly active users**, the package is a foundational tool in the R ecosystem. This level of usage—comparable to well-established data manipulation and statistical packages—justifies investment in modernising its infrastructure to integrate with contemporary R practices (lazy evaluation, functional programming, parallelisation).

Users across combinatorics research, statistics, operations research, and scientific computing depend on this package. Enabling standard iterator support will unlock new use cases and remove artificial barriers to productivity for researchers working with large combinatorial spaces.

Project Plan

Phase 0: Start-up (July 2026, Weeks 1–4)

- Establish collaboration infrastructure: GitHub issue tracking, pull request templates, contributor guidelines.
- Review existing `partitions` (Robin K. S. Hankin 2006) codebase: C API, R wrapper functions, test coverage, and GitHub open issues.
- Draft S3 `partitions_iter` class structure and method signatures.
- Set up continuous integration (CI) via GitHub Actions to ensure all changes pass `R CMD check` and `testthat` suite.
- Finalise scope and technical design with Robin Hankin.

Effort: 8 hours (Sam Kamperis), 8 hours (Robin Hankin)

Phase 1: Core Implementation (August 2026, Weeks 5–8)

- Implement `iparts()`, `irestrictedparts()`, `idiffparts()` factory functions.
- Implement `nextElem.partitions_iter()` method for each enumeration type.
- Ensure backward compatibility: existing `parts()`, `firstpart()`, `nextpart()` APIs remain unchanged.
- Write unit tests (`testthat` (Wickham 2011)) covering basic iteration, edge cases (n=1, n=0, m>n, etc.), and concurrent iterator coexistence.
- Robin Hankin: technical guidance and review (8 hours).

Effort: 16 hours (Sam Kamperis), 8 hours (Robin Hankin)

Phase 2: Integration & Testing (September 2026, Weeks 9–12)

- Integrate with `foreach` package: test that `foreach(part = iparts(n)) %do%` and `%dopar%` work correctly.
- Integrate with `itertools` (Weston and Wickham 2014) package: verify chaining via `itertools::ichain()`, filtering via `ifilter()`, etc.
- Address outstanding GitHub issues (bug fixes, documentation improvements) in the context of the iterator framework.

- Expand `testthat` suite to cover parallel iteration, GitHub issue resolution, and mathematical correctness benchmarks.

Effort: 16 hours (Sam Kamperis), 4 hours (Robin Hankin)

Phase 3: Vignette & Documentation (October 2026, Weeks 13–18)

- Write a detailed Quarto vignette (5+ worked examples):
  1. Serial enumeration: Basic `while` loop with `nextElem()`.
  2. `foreach()` parallelisation: Distributing partition enumeration across cores/cluster.
  3. `itertools` composition: Chaining, filtering, zipping partition iterators.
  4. Memory-efficiency comparison: Bulk generation (`parts(n)`) vs. lazy iteration (`iparts(n)`) for large n.
  5. Real-world use case: Sampling partitions, computing partition statistics, parallel optimisation.
- Update package `README.md` with modern iterator examples.
- Ensure all R documentation (`.Rd` files) reflects new iterator functions.
- Add CRAN-compliant licensing headers if necessary.
- Robin Hankin: two-day focused visit in Oxford (16 hours).

Effort: 16 hours (Sam Kamperis), 16 hours (Robin Hankin)

Phase 4: Review & Release (November 2026, Weeks 19–22)

- Code review with Robin Hankin; incorporate feedback.
- Final `R CMD check` and comprehensive testing.
- Prepare CRAN submission: ensure all checks pass, update `NEWS.md`, verify compatibility.
- Integrate 5-hour contingency for extended review, issue triage, or unexpected fixes.
- Robin Hankin: technical guidance and review (4 hours).

Effort: 16 hours (Sam Kamperis), 4 hours (Robin Hankin)

Phase 5: Publicity & Dissemination (December 2026, Weeks 23–26)

- Submit to CRAN.
- Address CRAN feedback (if any) and release new package version.
- Announce release on R Consortium blog and community channels.
- Robin Hankin: technical guidance and review (4 hours).

Effort: 8 hours (Sam Kamperis), 4 hours (Robin Hankin)

Budget & Funding Plan

Total Funding: $11,989.98 **Duration:** 6 months (1 July – 31 December 2026)
Total Hours: 124 (Sam Kamperis: 80, Robin Hankin: 44)

Phase	Dates	Sam Kamperis (hrs)	Robin Hankin (hrs)	Deliverables	Funding (%)
Phase 0: Start-up	July	8	8	Collaboration infra, design doc, CI setup	12%
Phase 1: Core Implementation	August	16	8	Iterator factory functions, unit tests, technical review	16%
Phase 2: Integration & Testing	September	16	4	`foreach` & `itertools` integration, expanded tests	16%
Phase 3: Vignette & Documentation	Oct	16	16	Comprehensive Quarto vignette, updated docs, Oxford visit	37%
Phase 4: Review & Release	Nov	16	4	CRAN submission, code review, contingency	17%
Phase 5: Publicity & Dissemination	Dec	8	4	Public announcement, CRAN feedback	8%
Total	July–Dec	**80**	**44**	Full modernisation + CRAN release	**100%**

Payment schedule: 50% upon contract signing, 50% upon completion. The contingency (5 hours) is included in Phase 4 for extended review or unexpected issues.

Other Aspects

Availability & Licensing

The `partitions` package is licensed under GPL-3. All code, tests, and documentation produced by this project will be released under the same licence, ensuring consistency and compatibility.

Code will be hosted on the official GitHub repository (github.com/RobinHankin/partitions) with pull requests and issues tracked transparently.

Publicity & Dissemination

- **ISC Blog Post (Mid-project, ~August 2026):** Summary of modernisation goals and progress.
- **Delivery Blog Post (Post-release, ~January 2027):** Technical introduction to the new iterator API, with code examples and benchmarks.
- **R Community Channels:** Announcements on R Weekly, R Discourse, and relevant social media.
- **Conference Consideration:** Abstract submission to useR! 2027 if timeline permits.

Success

Definition of Done

Success is measured by the following concrete, verifiable outcomes:

1. **Code merged and released:** All iterator implementation code is merged into the main branch of the `partitions` (Robin K. S. Hankin 2006) GitHub repository and accepted by Robin Hankin for inclusion in a new CRAN release (version number TBD, e.g., `partitions >= 1.11-0`).

2. **Standard iterator conformance:** The new `partitions_iter` class is fully compliant with the `iterators` (Analytics and Weston 2022) package S3 interface. Users can pass iterators directly to `foreach()` and `itertools` (Weston and Wickham 2014) functions without custom wrappers.

3. **Comprehensive test coverage:** A full `testthat` (Wickham 2011) suite validates:

   - Correctness: iterators produce identical results to bulk-generation functions.
   - Concurrency: multiple iterators coexist and advance independently.
   - Edge cases: n=0, n=1, m>n, restrictedparts with various constraints.
   - Interoperability: `foreach()`, `itertools::ichain()`, `itertools::ifilter()`, and parallel backends.

4. **Published vignette:** A detailed Quarto vignette is included in the package, demonstrating 5+ worked examples spanning serial iteration, parallelisation, composition, and real-world use cases.

5. **GitHub issues resolved:** Outstanding GitHub issues relevant to iteration are addressed and closed (or documented as out-of-scope with clear justification).

Measuring Success

We will track the following metrics throughout the project:

Metric	Success Criterion	Measurement Method
Code Coverage	90% of new iterator code covered by tests	`covr` (Hester 2025) package report
CRAN Compliance	All `devtools::check()` and `R CMD check` tests pass with zero errors/warnings	CI pipeline; manual CRAN dry-run
Interoperability	`foreach()` and `itertools` examples execute without error	Unit tests + manual vignette examples
Memory Efficiency	Lazy iteration peak memory is O(n) per step; bulk generation materialises the full partition matrix	`bench::mark(memory = TRUE)` (Hester and Vaughan 2023) comparing `parts(50)` vs. `iparts(50)`; reproducible scripts in `inst/benchmarks/`
Documentation Quality	Vignette is comprehensible to users unfamiliar with iterators	User testing (informal review by collaborators)
GitHub Issues	80% of open iterator-related issues resolved or documented	GitHub issue tracker review
CRAN Release	New version submitted and accepted by CRAN	Official email confirmation from CRAN maintainers

Progress Checkpoints

- **End of July (Milestone 1):** Design doc approved by Robin; CI infrastructure operational; initial test suite in place.
- **End of September (Milestone 2–3):** `iparts()`, `irestrictedparts()`, `idiffparts()` factory functions implemented and tested; `foreach()` integration working.
- **End of October (Milestone 3):** `itertools` integration functional; GitHub issues triaged and addressed.
- **End of November (Milestone 4):** Vignette drafted and reviewed; all documentation updated.
- **End of December (Milestone 5):** CRAN submission completed; public announcement released.

Future Work

Short-term (Post-Release)

- **Additional partition types:** Extend iterator support to `setparts()`, `blockparts()`, `compositions()` if demand warrants.
- **Performance optimisation:** Profile and optimise the `nextElem()` closure if memory/CPU overhead becomes significant in high-throughput scenarios.
- **Educational resources:** Publish a blog post or tutorial on the `partitions` package ecosystem for the R community blog.

Medium-term (6–12 Months)

- **Parallel sampling:** Develop advanced itertools, e.g., `isample_partitions()` for unbiased random partition sampling in parallel.
- **Integration with `furrr` (Vaughan and Dancho 2022):** Ensure seamless compatibility with `furrr::future_map()` and other functional programming packages.
- **Package ecosystem integration:** Collaborate with authors of related packages (e.g., combinatorics, optimisation) to showcase shared iterator workflows.

Long-term (1–2 Years)

- **Persistent state storage:** Support serialisation of iterator state to disk, enabling resumption of long-running enumerations across sessions.
- **Distributed computing:** Explore Apache Spark / Hadoop integration for massive-scale partition enumeration in clusters beyond standard R parallelism.
- **Combinatorics library expansion:** Consider generalising the iterator pattern to other combinatorial enumerations (permutations, combinations, Young tableaux, etc.) as a separate infrastructure project.

Analytics, Revolution, and Steve Weston. 2022. *Iterators: Provides Iterator Construct*. https://doi.org/10.32614/CRAN.package.iterators.

Hester, Jim. 2025. *Covr: Test Coverage for Packages*. https://doi.org/10.32614/CRAN.package.covr.

Hester, Jim, and Davis Vaughan. 2023. *Bench: High Precision Timing of R Expressions*. https://doi.org/10.32614/CRAN.package.bench.

Lai, Randy. 2020. *Arrangements: Fast Generators and Iterators for Permutations, Combinations, Integer Partitions and Compositions*. https://doi.org/10.32614/CRAN.package.arrangements.

Robin K. S. Hankin. 2006. "Additive integer partitions in R." *Journal of Statistical Software, Code Snippets* 16 (May).

Vaughan, Davis, and Matt Dancho. 2022. *Furrr: Apply Mapping Functions in Parallel Using Futures*. https://doi.org/10.32614/CRAN.package.furrr.

Weston, Steve, and Hadley Wickham. 2014. *Itertools: Iterator Tools*. https://doi.org/10.32614/CRAN.package.itertools.

Wickham, Hadley. 2011. "Testthat: Get Started with Testing." *The R Journal* 3: 5–10. https://journal.r-project.org/articles/RJ-2011-002/.

Wood, Joseph. 2026a. *High Performance Benchmarks*. Vignette for R package RcppAlgos. https://cran.r-project.org/web/packages/RcppAlgos/vignettes/HighPerformanceBenchmarks.html.

Wood, Joseph. 2026b. *RcppAlgos: High Performance Tools for Combinatorics and Computational Mathematics*. https://jwood000.github.io/RcppAlgos/.